Deep Reinforcement Learning

Dynamic Programming

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

Content

Content

- What is Dynamic Programming (DP)?
- The two central tasks in RL
- Policy evaluation
- Policy improvement
- Policy iteration
- Value iteration
- Asynchronous DP
- Generalized policy iteration

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1	Multi-armed bandits	Exploration-exploitation dilemma
2	Markov decision processes	Dynamics, rewards, policies
3	Dynamic programming	Optimal decision making with *full knowledge*
4	Monte Carlo methods	
5	Temporal difference learning & Q-learning	
	Deep-learning-based methods	
	Model-Based Control	
	Advanced Topics	

Lecture contents

What is Dynamic Programming (DP)?

What is Dynamic Programming (DP)?

Dynamic Programming (**DP**) refers to two central factors:

- *Dynamic*: a sequential or temporal problem structure
- *Programming*: mathematical optimization, i.e., an algorithmic / a numerical solution

Further characteristics:

- DP is a collection of algorithms to solve MDPs and neighboring problems.
  - We will **focus only on finite MDPs**.
  - In case of continuous action/state space: apply quantization.
- Use of value functions to organize and structure the search for an optimal policy.
- Breaks problems into subproblems and solves them.

Bellman's optimality principle (1)

We here consider finite state and action spaces $\mathcal{S}$ and $\mathcal{A}$. In the last lecture, we have learned about the *return*, the *value function* and the *Bellman equation*

$$
\begin{aligned}
g_t &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \\
V^{\pi}(s_t) &= \mathbb{E}_{\pi}\left[g_t | s_t\right] = \mathbb{E}\left[r_t | s_t\right] + \gamma \mathbb{E}_{\pi}\left[V(s_{t+1}) | s_t\right], \\
&= \mathbb{E}_{\pi}\left[r_t + \gamma V^{\pi}(s_{t+1}) | s_t\right]. \qquad (1)
\end{aligned}
$$

Theorem: Bellman's principle of optimality (Bellman 1954)
An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

In other/modern words: "the tail of an optimal sequence is optimal for the tail subproblem" (Bertsekas 2019)

Bellman's optimality principle (2)

From this, we derived the *Bellman optimality equation* (Bellman 1954):

$$\begin{aligned} V^*(s) &= \max_{a \in \mathcal{A}} \mathbb{E}\left[r_t + \gamma V^*(s_{t+1}) | s_t = s, a_t = a\right] \\ &= \max_{a \in \mathcal{A}} \sum_{s_{t+1} \in \mathcal{S}} p\left(s_{t+1} | s_t = s, a_t = a\right) \left[r_t + \gamma V^*(s_{t+1})\right]. \end{aligned} \tag{2}$$

Similarly, we can derive an optimal Q-function:

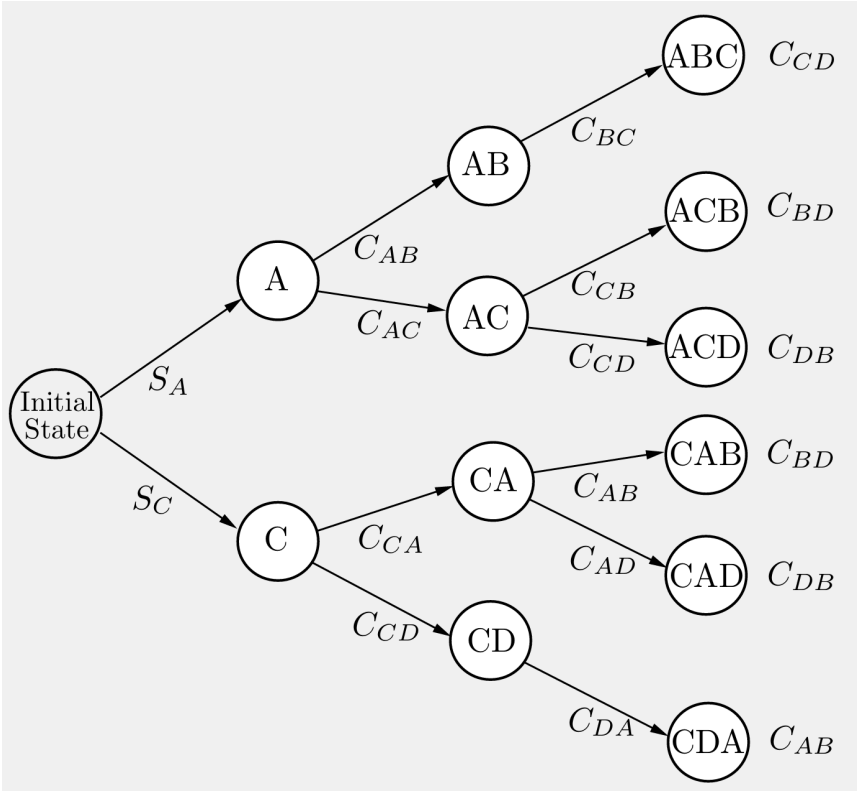
$$\begin{aligned} Q^*(s,a) &= \mathbb{E}\left[r_t + \gamma \max_{a' \in \mathcal{A}} Q^*(s_{t+1}, a') \middle| s_t = s, a_t = a\right] \\ &= \sum_{s_{t+1} \in \mathcal{S}} p\left(s_{t+1} | s_t = s, a_t = a\right) \left[r_t + \gamma \max_{a' \in \mathcal{A}} Q^*(s_{t+1}, a')\right]. \end{aligned} \tag{3}$$

Central idea of DP: turn these equations into algorithmic assignments.

A DP example from production (1)

- to produce a certain product, four operations (denoted by A, B, C, and D) must be performed on a certain machine.
- We assume that operation B can be performed only after operation A has been performed, and operation D can be performed only after operation C has been performed. *(Thus the sequence $CDAB$ is allowable but the sequence $CDBA$ is not.)*
- The setup cost C_{mn} for passing from any operation m to any other operation n is given.
- There is also an initial startup cost S_A or S_C for starting with operation A or C, respectively.
- The cost of a sequence is the sum of the setup costs associated with it; for example, the operation sequence $ACDB$ has cost

$$S_A + C_{AC} + C_{CD} + C_{DB}.$$



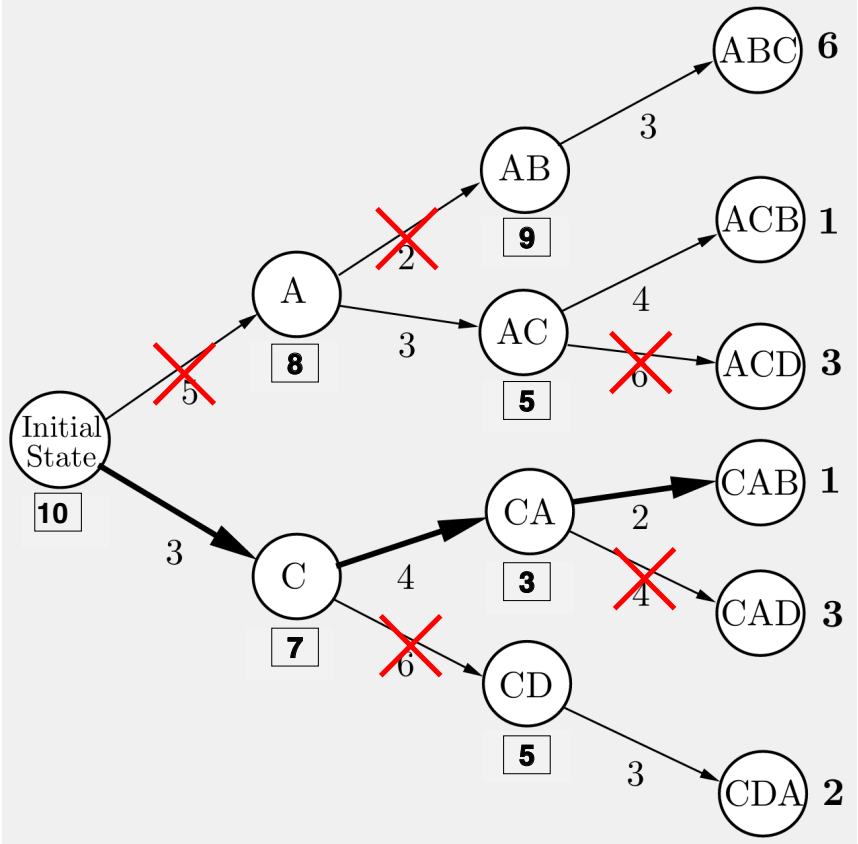
Question: In this example, what is (in the context of RL)

- the state s?
- the action a (and action set $\mathcal{A}(s)$)?

Example taken from (Bertsekas 2019, Example 1.1.1)

A DP example from production (2)

- According to the principle of optimality, the “tail” portion of an optimal schedule must be optimal.
- For example, suppose that the optimal schedule is $CABD$.
- Then, having scheduled first C and then A, it must be optimal to complete the schedule with BD rather than with DB.
- With this in mind, we solve
  - all possible tail subproblems of length two,
  - then all tail subproblems of length three,
  - and finally the original problem that has length four.
  - *(Subproblems of length one are trivial because there is only one unscheduled operation.)*



The two central tasks in RL

The two central tasks in RL

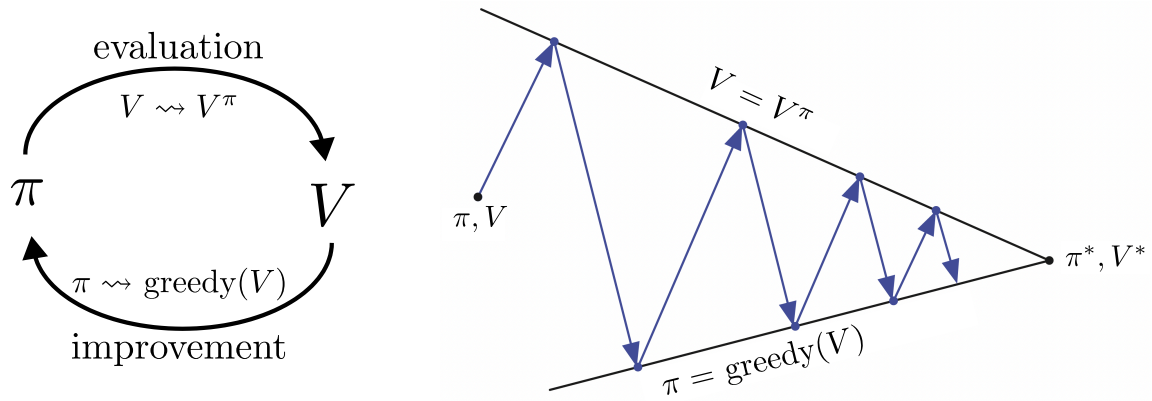
In all chapters, we will distinguish between **two core learning tasks in reinforcement learning**:

1. **The prediction problem** (*evaluation*)

- For a fixed policy π, we want to **estimate the Value function** $V^{\pi}(s)$ (or the Q function $Q^{\pi}(s, a)$)
- The methods vary strongly in terms of the knowledge we have, e.g.,
  - We know p: Dynamic programming.
  - We don't know p: Learning from experience (Monte Carlo, temporal difference learning, …).
  - Many variations and combinations exist!

2. **The control problem** (*improvement*)

- We want to **improve our policy** π towards the optimal π^* that maximizes the value function.
- This usually requires a back-and-forth between 1. and 2. (known as *generalized policy iteration (GPI)*).
  - Given our estimate of V^{π} / Q^{π}, we improve the policy: $\pi' > \pi$.
  - This invalidates V^{π} / Q^{π} $\Rightarrow$ update estimates: $V^{\pi'}$ / $Q^{\pi'}$.
  - We then further improve $\pi'' > \pi'$, and so on.



Generalized Policy Iteration (GPI) (Sutton and Barto 1998)

Policy evaluation

The prediction problem

Let's start with the *prediction problem*: For a given policy π, how do we compute V^π?

Let's revisit (1):

$$V^\pi(s) = \mathbb{E}_\pi \left[r + \gamma V^\pi(s') | s \right] = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p\left(s'|s,a\right) \left[r + \gamma V^\pi(s') \right] \quad (4)$$

- If the dynamics p are known, (4) is a system of $|S|$ linear equations in $|S|$ unknowns (the $V^\pi(s)$, $s \in \mathcal{S}$) $\Rightarrow$ Solution is straightforward, if tedious.
- Iterative solution method:
  - Consider a sequence of approximate value functions $V_0, V_1, V_2, \ldots$
  - Choose V_0 arbitrarily (except that the terminal state, if any, must be given value 0).
  - Each successive approximation is obtained by using the Bellman equation (4) as an update rule.

💡 Existence and uniqueness of V^π are guaranteed as long as either $\gamma < 1$ or eventual termination is guaranteed from all states under the policy π.

Iterative policy evaluation

Use the Bellman equation (4) as an update rule:

$$V_{k+1}(s) = \mathbb{E}_\pi \left[r + \gamma V_k(s') | s \right] = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p\left(s'|s,a\right) \left[r + \gamma V_k(s') \right] \qquad (5)$$

- $V_k = V^\pi$ is a fixed point for this update rule: (5) is satisfied for V^π.
- The sequence $\{V_k\}$ converges to V^π as $k \to \infty$ (under the same conditions guaranteeing existence of V^π).
- This algorithm is called **iterative policy evaluation**:
  - For each state s, replace the old value of s with a new value …
  - … obtained from the old values at the successor states s' and the expected immediate rewards.
  - This is called an *expected update*.
- *Simplest implementation*: keep two copies of V (i.e., V_k and V_{k+1}), then sweep over $\mathcal{S}$.

In-place iterative policy evaluation

A more memory-efficient version (that tends to converge faster): in-place updates!

Algorithm: In-place iterative policy evaluation for estimating $V \approx V^{\pi}$.

Input: policy π

Parameters: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$

while (True):

$\Delta \leftarrow 0$

for $s \in \mathcal{S}$:

$V_{\text{old}}(s) \leftarrow V(s)$

$V(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s,a) \left[r + \gamma V(s') \right]$

$\Delta \leftarrow \max(\Delta, |V_{\text{old}}(s) - V(s)|)$

if $\Delta < \theta$ **then** STOP

A remark on *bootstrapping*

Many of you will know bootstrapping from **supervised learning**:

Bootstrapping in *supervised learning*

- We have a training dataset $\mathcal{D}$ comprised of N input-output tuples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$.
- We would like to improve our model by training on multiple datasets, but we don't have more data.
- To this end, we create an *ensemble* of M datasets $\{\tilde{\mathcal{D}}_j\}_{j=1}^{M}$ by drawing N samples i.i.d.* from $\mathcal{D}$ *with replacement*

$$(\tilde{x}_{j,1}, \tilde{y}_{j,1}), (\tilde{x}_{j,2}, \tilde{y}_{j,2}), \ldots, (\tilde{x}_{j,N}, \tilde{y}_{j,N}) \sim \mathcal{D}.$$

In **reinforcement learning**, bootstrapping has a different meaning:

Bootstrapping in *reinforcement learning*

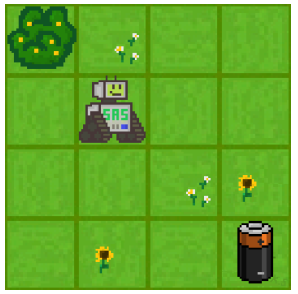
- In order to update an estimate of a function of interest (say, $V(s)$ or $Q(s,a)$), *we rely on yet another estimate*.
- We have seen this just a minute ago! In the (iterative) policy evaluation algorithm, we have

$$V(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p\left(s'|s,a\right) \left[r + \gamma V(s')\right].$$

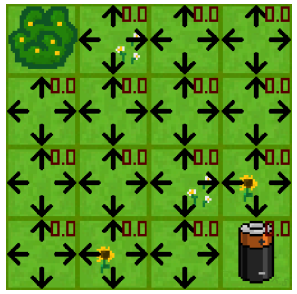
* i.i.d. = independent and identically distributed.

Example: Gridworld

We have a small robot in a gridworld that wants to recharge.

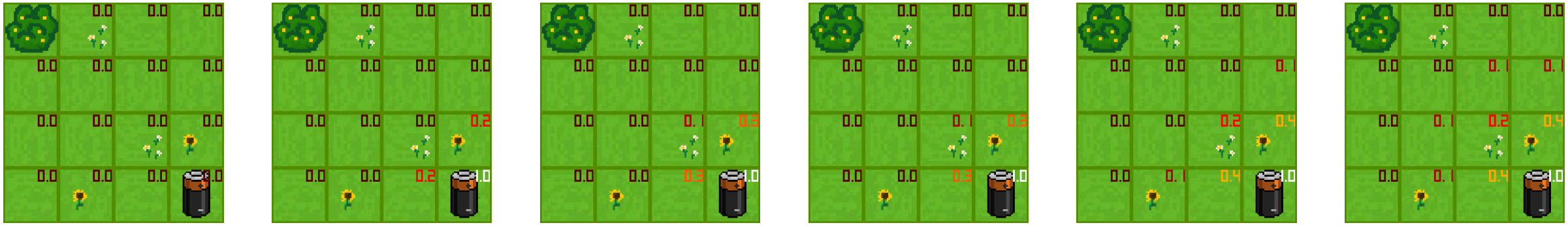


Gridworld



Random policy

- **Initial state s_0**: a random valid field.
- **Goal**: reach the battery ($r = 1$, otherwise $r = 0$).
- $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ (*leaving* or *invalid field* $\Rightarrow$ no movement).
- $\pi(\cdot|s) = [0.25, 0.25, 0.25, 0.25]^\top \; \forall \, s \in \mathcal{S}$.
- **Discount factor**: $\gamma = 0.8$.
- **Terminal state**: If the robot hits the battery, it receives $r = 1$ (potentially discounted) and the episode ends.
- **Optimal strategy**: Move towards the battery as quickly as possible.



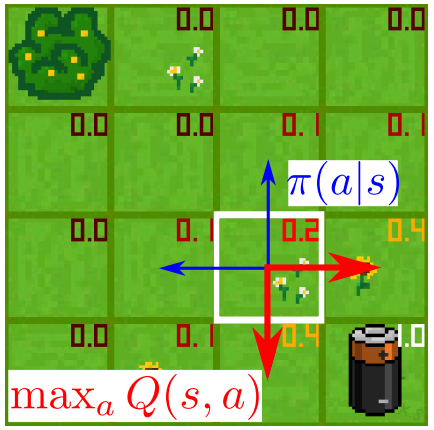
$k = 0$ $k = 1$ $k = 2$ $k = 3$ $k = 10$ $k = 1000$

In-place iterative policy evaluation

Policy improvement

Now that we can learn a policy, how do we improve it?

- Assume that we have used the policy evaluation algorithm to estimate V^π.
- How can this help us for finding a better behavior?
- Choose the next action freely (that is, $a \neq \pi(s)$)



$k = 1000$

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[r + \gamma V^\pi(s') | s, a \right]$$
$$= \sum_{s' \in \mathcal{S}} p\left(s'|s,a\right) \left[r + \gamma V^\pi(s') \right].$$

$\Rightarrow$ If $Q^\pi(s,a) > V^\pi(s)$, then choosing a over $\pi(s)$ in state s yields a better policy.
$\Rightarrow$ This is a special instance of the **policy improvement theorem**.

We're here using some arguments for deterministic policies (i.e., $a = \pi(s)$), but the arguments also hold in the probabilistic setting.

The policy improvement theorem

Theorem: Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

$$\sum_{a \in \mathcal{A}} \pi'(a|s) Q^{\pi}(s,a) \geq V^{\pi}(s).$$

Then $V^{\pi'}(s) \geq V^{\pi}(s)$ for all $s \in \mathcal{S}$.

Proof:

$$V^{\pi}(s_t) \leq \sum_{a_t \in \mathcal{A}} \pi'(a_t|s_t) Q^{\pi}(s_t,a_t) = \sum_{a_t \in \mathcal{A}} \pi'(a_t|s_t) \sum_{s_{t+1} \in \mathcal{S}} p\left(s_{t+1}|s_t,a_t\right) \left[r_t + \gamma V^{\pi}(s_{t+1})\right] = \mathbb{E}_{\pi'}\left[r_t + \gamma V^{\pi}(s_{t+1})|s_t\right] \quad (6)$$

Since this holds for all $s \in \mathcal{S}$, we can apply the same procedure at $s_{t+1} \Rightarrow V^{\pi}(s_{t+1}) \leq \mathbb{E}_{\pi'}\left[r_{t+1} + \gamma V^{\pi}(s_{t+2})|s_{t+1}\right]$.

Substitute into (6): $V^{\pi}(s_t) \leq \mathbb{E}_{\pi'}\left[r_t + \gamma \mathbb{E}_{\pi'}\left[r_{t+1} + \gamma V^{\pi}(s_{t+2})|s_{t+1}\right]|s_t\right]$

Using the linearity of expectations and the law of total expectation ($\mathbb{E}\left[\mathbb{E}\left[x|y\right]\right] = \mathbb{E}\left[x\right]$; see, e.g., Wikipedia):

$$V^{\pi}(s_t) \leq \mathbb{E}_{\pi'}\left[r_t + \gamma r_{t+1} + \gamma^2 V^{\pi}(s_{t+2})|s_t\right].$$

Repeat infinitely often: $V^{\pi}(s_t) \leq \mathbb{E}_{\pi'}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \ldots|s_t\right] = V^{\pi'}(s_t)$ □

Greedy policy improvement

- Given a policy π and its value function V^π, we can easily evaluate a change in the policy at a single state.
- Natural extension: at each state, select the action that appears best according to $Q^\pi(s,a)$.
- In other words, to consider the new **greedy policy** π', given by

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} Q^\pi(s,a) = \arg\max_{a \in \mathcal{A}} \mathbb{E}_\pi \left[r + \gamma V^\pi(s') | s, a \right] = \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p\left(s'|s,a\right) \left[r + \gamma V^\pi(s') \right],$$

  with ties broken arbitrarily.
- Improving on a policy by making it greedy with respect to the value function V^π is called **policy improvement**.

When do we stop?

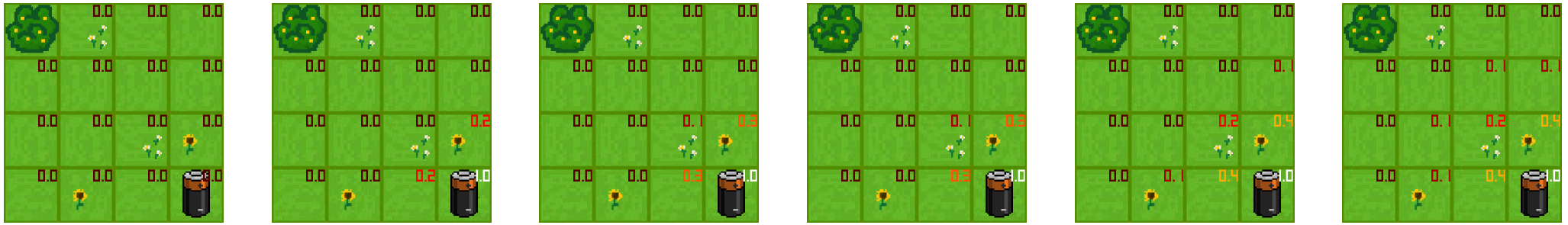
- Suppose π' is as good as π, but not better. That is, $\pi'(s) = \pi(s)$ for all $s \in \mathcal{S}$.
- Then π' satisfies the Bellman optimality condition (2):

$$V^{\pi'}(s) = \max_{a \in \mathcal{A}} \mathbb{E}_\pi \left[r + \gamma V^{\pi'}(s') \middle| s, a \right] = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p\left(s'|s,a\right) \left[r + \gamma V^{\pi'}(s') \right],$$

Conclusion: Policy improvement yields a strictly better policy, unless π is already optimal!

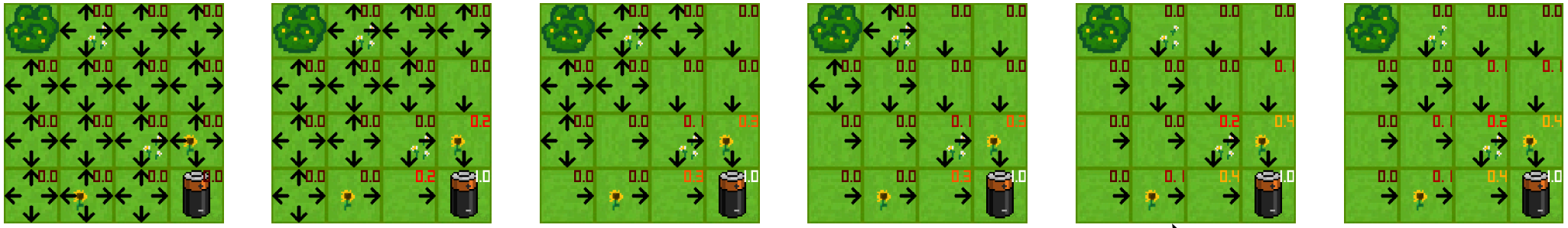
Example: Gridworld

V_k^π for the random policy



$k = 0$ $k = 1$ $k = 2$ $k = 3$ $k = 10$ $k = 1000$

Greedy policy w.r.t. V_k^π



Random policy
$\pi(a|s) = [\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}]^\top$

Optimal policy

💡 in the stochastic setting, each maximizing action can be given a non-zero probability in $\pi(a|s)$. Any apportioning scheme is allowed as long as all submaximal actions are given zero probability.

Policy and value iteration

Policy iteration (1)

- Assume our policy π has been improved using V^{π} to yield a better policy π'
- We can compute $V^{\pi'}$ and improve it again to yield an even better π''.
- We thus obtain a sequence of monotonically improving policies and value functions:

$$\pi_0 \xrightarrow{E} V^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V^{\pi_1} \ldots \xrightarrow{I} \pi^* \xrightarrow{E} V^* = V^{\pi^*},$$

  where $\xrightarrow{E}$ denotes a policy evaluation and $\xrightarrow{I}$ denotes a policy improvement.
- Each policy is guaranteed to be a strict improvement over the previous one (unless it is already optimal).
- Finite MDPs have a finite number of deterministic policies: convergence to π^* and V^* in a finite number of iterations.

> This way of finding an optimal policy is called **policy iteration**.

💡 since each policy evaluation is itself an iterative computation, we start it with the value function for the previous policy. This typically results in a great increase in the speed of convergence of policy evaluation (presumably because the value function changes little from one policy to the next).

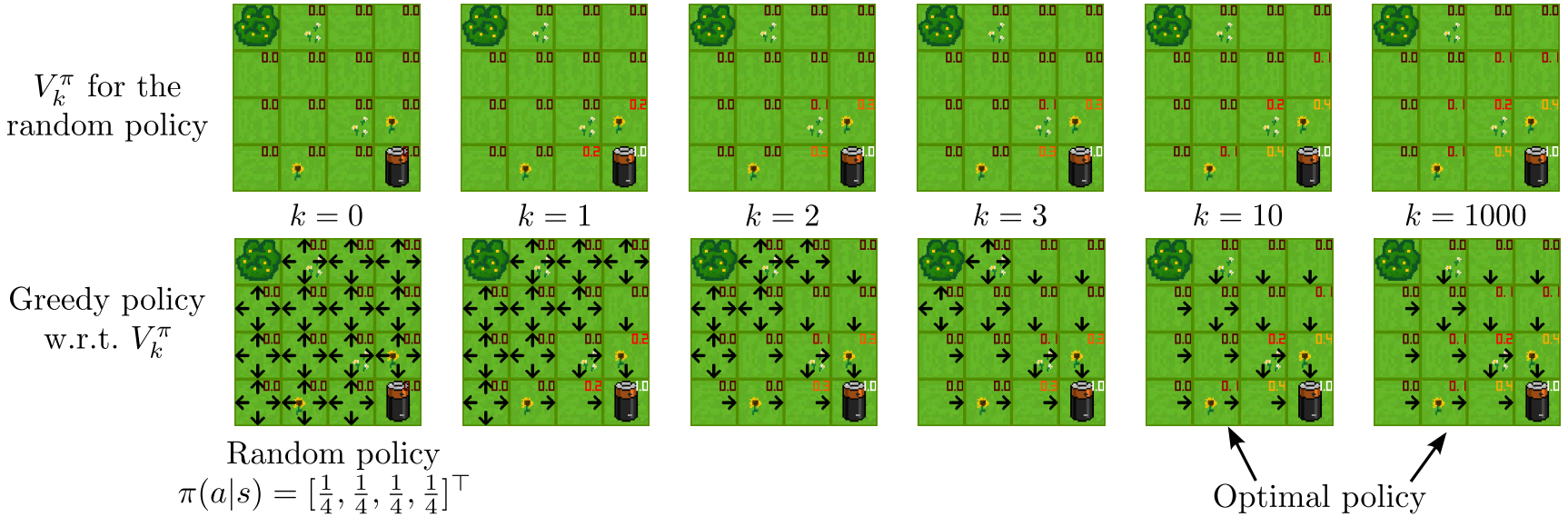
Policy iteration (2)

Algorithm: Policy Iteration for estimating $\pi \approx \pi^*$.

1. **Initialization**: $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}$ arbitrarily for all $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$, small threshold $\theta > 0$
2. **Policy evaluation**:
   while (True):
   $\Delta \leftarrow 0$
   for $s \in \mathcal{S}$:
   $V_{\text{old}} \leftarrow V(s)$
   $V(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s,a) [r + \gamma V(s')]$
   $\Delta \leftarrow \max(\Delta, |V_{\text{old}} - V(s)|)$
   if $\Delta < \theta$ **then** STOP
3. **Policy improvement**:
   $\textbf{flag}_{\text{stable}} = true$
   for $s \in \mathcal{S}$:
   $\pi_{\text{old}}(s) \leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s,a) [r + \gamma V^{\pi}(s')]$
   if $\pi(s) \neq \pi_{\text{old}}(s)$ **then** $\textbf{flag}_{\text{stable}} = false$
   if $\textbf{flag}_{\text{stable}} = true$ **then** return $V \approx V^*$ and $\pi \approx \pi^*$ **else** go back to **2. Policy evaluation**

Value iteration (1)

- What's the main drawback of policy iteration?
  $\Rightarrow$ it's expensive to solve the policy evaluation loop in each iteration (nested loops!)
- Do we have to wait until convergence of V_k to V^π every time?
  $\Rightarrow$ the gridworld example suggested no!



- **Value iteration**: truncate the policy evaluation after **one** step!

Value iteration (2)

Let's look at our earlier derivation of the policy evaluation procedure:

$$V_{k+1}(s) = \mathbb{E}_{\pi}\left[r + \gamma V_k(s')|s\right] = \sum_{a\in\mathcal{A}} \pi\left(a|s\right) \sum_{s'\in\mathcal{S}} p\left(s'|s,a\right)\left[r + \gamma V_k(s')\right] \tag{5}$$

Now, we change this to directly update the value function using the maximizing action (policy improvement step):

$$V_{k+1}(s) = \max_{a\in\mathcal{A}} \mathbb{E}\left[r + \gamma V_k(s')|s,a\right] = \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} p\left(s'|s,a\right)\left[r + \gamma V_k(s')\right] \tag{7}$$

Let's compare this to the Bellman equation:

$$V^*(s) = \max_{a\in\mathcal{A}} \mathbb{E}\left[r + \gamma V^*(s')|s,a\right] = \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} p\left(s'|s,a\right)\left[r + \gamma V^*(s')\right]. \tag{2}$$

$\Rightarrow$ We have simply turned (2) into an iterative procedure (using *bootstrapping*)!

$\Rightarrow$ Value iteration can be shown to converge for arbitrary V_0

Value iteration (3)

Algorithm: Value iteration for estimating $\pi \approx \pi^*$.

Parameters: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$

while $(\Delta > \theta)$:

$\Delta \leftarrow 0$

for $s \in \mathcal{S}$:

$V_{\text{old}}(s) \leftarrow V(s)$

$V(s) \leftarrow \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p\left(s' | s, a\right) \left[r + \gamma V(s')\right]$

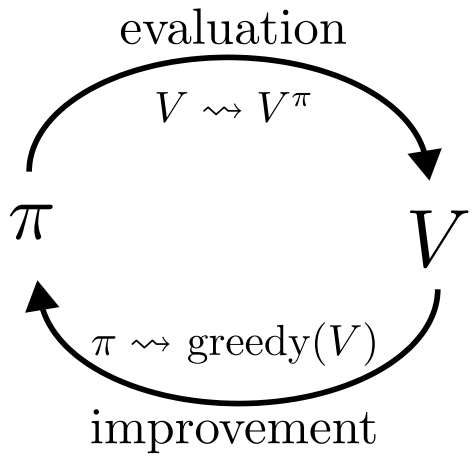
$\Delta \leftarrow \max(\Delta, |V_{\text{old}}(s) - V(s)|)$

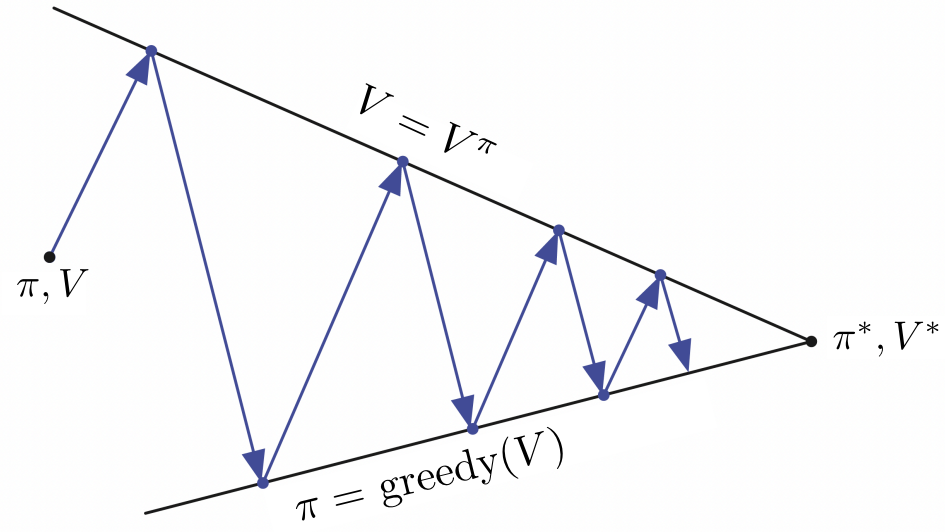
if $\Delta < \theta$ **then** STOP

Generalized policy iteration

Generalized policy iteration (GPI)

- Almost all RL algorithms can be summarized under the GPI framework.
- Back-and-forth between two interacting processes:
  - making the value function consistent with the current policy (policy evaluation) $\Rightarrow V^{\pi}$,
  - greedy policy w.r.t. the current value function (policy improvement) $\Rightarrow \pi = \text{greedy}(V)$.
- Convergence towards Bellman optimality
  - value function stabilizes when it's consistent with the current policy,
  - the policy stabilizes when it is greedy w.r.t. the current value function.
  - stabilization when a policy has been found that is greedy w.r.t. its own evaluation function.





Generalized Policy Iteration (GPI) (Sutton and Barto 1998)

Summary / what you have learned

Summary / what you have learned

- **Dynamic Programming**
  - is a useful technique for solving decision making problems,
  - is based on Bellman's optimality principle,
  - is limited in problem size (**curse of dimensionality**).
- **Policy evaluation** & **policy improvement** are iterative procedures to
  - approximate the value function V^{π} for a given policy π,
  - make a greedy upgrade for the policy π given a value function V.
- **Policy iteration** alternates between policy evaluation and improvement to find an optimal policy π^*.
- **Value iteration** is an improved version where we intertwine policy evaluation & improvement more closely.
- **Generalized Policy Iteration** (**GPI**)
  - is the general framework describing various versions of the above iterative process,
  - Almost all RL algorithms can be interpreted as versions of GPI.

References

Bellman, Richard. 1954. “The Theory of Dynamic Programming.” *Bulletin of the American Mathematical Society* 60 (6). American Mathematical Society (AMS): 503–15. doi:10.1090/s0002-9904-1954-09848-8.

Bertsekas, Dimitri. 2019. *Reinforcement Learning and Optimal Control*. Vol. 1. Athena Scientific.

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.